

CHATDB：用数据库作为其 符号内存

Chenxu Hu^{1*} Jie Fu^{2*†} Chenzhuang Du¹ Simian Luo¹ Junbo Zhao³ Hang Zhao^{(1)†}

¹清华大学 ²北京人工智能学会

³浙江大学

fujie@baai.ac.cn hangzhao@mail.tsinghua.edu.cn

摘要

具有内存的大型语言模型（LLM）在计算上是通用的（Schuurmans, 2023 年）。然而，主流 LLM 并没有充分利用内存，其设计深受生物大脑的影响。由于其近似性和容易积累错误的特点，传统的神经记忆机制无法支持 LLMs 模拟复杂推理。在本文中，我们从现代计算机体系结构中寻找灵感，用符号记忆来增强 LLM，从而实现复杂的多跳推理。这种符号记忆框架被实例化为一个 LLM 和一组 SQL 数据库，其中 LLM 生成 SQL 指令来操作 SQL 数据库。我们在一个需要复杂推理的合成数据集上验证了所提出的内存框架的有效性。项目网站：<https://chatdatabase.github.io/>。

项目网站：<https://chatdatabase.github.io/>。

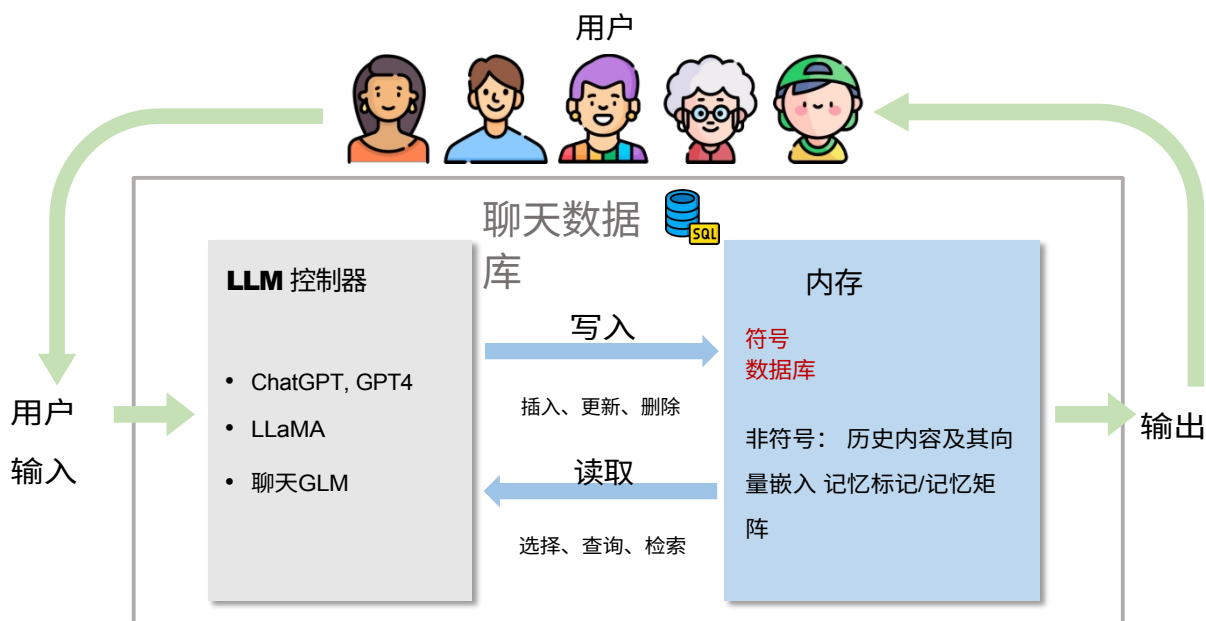


图 1：ChatDB 的整体工作流程。LLM 控制器控制对内存的读写操作。内存存储历史信息，并提供相关历史信息以协助响应用户输入。在 ChatDB 中，我们的重点是将数据库作为 LLM 的符号存储器来增强 LLM。

*同等技术贡献。

†通讯作者。

1 简介

大型语言模型 (LLMs)，如 GPT-4 (OpenAI, 2023 年) 和 PaLM 2 (Anil 等, 2023 2023 年)，已日益成为现代人工智能 (AI) 系统的重要组成部分，彻底改变了我们对自然语言处理 (NLP) 的理解，并改变了各行各业 (郝等, 年2023 年; 王等,)。虽然 LLM 在理解和生成与语境相关的反应方面取得了长足进步，但它们也有局限性 (Chen 等人, 2023 年)。主要挑战之一是与语言模型的多轮交互会产生大量标记，这很容易超出 LLM 的输入标记限制。例如，GPT-4 (32K) 只能处理 32,000 个标记。随着交互的进行，LLM 必须维护上下文信息 (如用户输入和之前的响应)，并根据积累的数据生成响应。然而，简单地将所有上下文信息串联起来并塞入 LLM，很容易超出 LLM 的处理能力并积累错误，从而导致模型跟不上对话的进程，并产生不够准确的回复。

人们已经探索了一些神经记忆机制 (Wu 等人, 2022a; Khattab 等人, 2022; Zhong 等人, 2022)，以克服 LLMs 标记输入有限的问题。记忆组件是存储和检索以往交互中相关信息的系统。然而，用传统的神经记忆增强 LLM 通常会导致在记忆中存储、检索和操作历史信息的困难，特别是对于需要复杂多跳推理的任务。两个主要原因是：(a) 它们不能以结构化的形式存储历史信息；(b) 它们对存储在内存中的信息的操作不是符号化的，因为它们都依赖于一些向量相似性计算，而这些计算可能是不准确的，从而导致错误的积累。

为了解决上述问题，我们建议使用数据库作为 LLM 的新型符号存储器。整个框架被命名为 ChatDB。如图 1 所示，ChatDB 由两部分组成：LLM 控制器及其内存。LLM 控制器可以是任何常用的 LLM (OpenAI, 2023; Touvron 等人, 2023; Du 等人, 2022; Zeng 等人, 2022)，负责控制内存的读写操作。LLM 的存储器可以是符号存储器或非符号存储器，也可以是两者的组合，负责存储历史信息，并在需要时提供信息，协助 LLM 响应用户输入。在 ChatDB 中，我们专注于使用数据库作为符号存储器，通过执行符号语言 (即 SQL 语句) 来结构化存储历史信息。这些 SQL 语句由 LLM 生成。在需要精确记录、修改、查询、删除和分析历史数据的场景中，将数据库作为符号存储器特别有用。例如，商店经理需要维护每天的销售记录，而使用纯文本或矩阵作为内存是不合适的 (Chen 等人, 2023 年)。然而，使用数据库作为外部符号存储器则非常合适。数据库可以使用 SQL 语句进行精确操作，包括数据插入、删除、更新和选择。因此，使用数据库作为外部符号存储器可确保管理和处理历史数据的精确性和效率，从而在需要高精度和长期数据记录与处理的场景中显著提高 LLM 的性能。

在 ChatDB 框架中，我们提出了内存链 (CoM) 方法，以更有效地操作外部符号内存，从而进一步增强 LLM 的推理能力。内存链方法将用户输入转化为一系列中间内存操作步骤，最终产生结果。通过内存链方法，一个复杂的问题被分解成多个内存操作步骤，从而大大降低了解决问题的复杂性。在 ChatDB 中，每个中间步骤都涉及一条或多条 SQL 语句。

我们的 ChatDB 在 LLM 领域做出了多项贡献。首先，我们建议将数据库作为 LLM 的外部符号存储器来增强 LLM，从而实现历史数据的结构化存储，并使用 SQL 语句进行符号化和复杂的数据操作。其次，我们的内存链方法通过将用户输入转换为多步中间内存操作，实现了有效的内存操作，从而提高了 ChatDB 的性能，使其能够处理复杂的多表数据库交互，并提高了准确性和稳定性。最后，我们的实验证明，用符号内存增强 LLM 可以提高多跳推理能力，防止错误积累，从而使 ChatDB 在合成数据集上的性能大大超过 ChatGPT。

2 相关工作

记忆增强大型语言模型 (LLMs)。 GPT-4 (OpenAI, 2023 年) 和 PaLM 2 (Anil 等人, 2023 年) 等 LLM 已展示出强大的推理和决策能力。然而，LLM 通常受限于其有限的上下文窗口大小 (例如，GPT-4 只能处理 32K 标记)。记忆增强型 LLM (Wu 等人, 2022a,b; Zhong 等人, 2022; Lewis 等人Guu 等, 2020; 人Park 等人, 2020; , 2023; Khattab 等人, 2022; Izacard 等人, 2022) 集成了一个记忆模块，可防止模型遗忘关键信息，并允许其处理超出上下文窗口大小的长文本输入。检索增强型上下文学习 (Khattab 等人, 2022 年)

在这种情况下，代理可以使用检索模型（RM）来检索相关信息，并将其作为提示插入到 LLM 中。例如，Auto-GPT³和 Generative Agents（Park 等人，2023 年）利用内存模块直接存储文本提示，使代理可以跟踪其历史。然后将过去和当前的提示输入 LLM 进行处理。神经图灵机（NMT）（Graves et al.）门控图序列神经网络（GGS-NN）（Johnson，2017 年）可构建和修改图形，并利用图形产生合理的输出。递归记忆变换器（RMT）（Bulatov 等人，2022 年）在输入和输出序列中引入额外的记忆标记，以存储、处理和交换长序列片段之间的局部和全局信息，然后训练模型来控制记忆操作和序列表征处理。

常识分子的推理。众所周知，LLM 在复杂的推理任务中很难胜任。以往的方法主要是结合专门设计的监督信号或微调来提高语言模型的推理能力（Piekos 等人，2021；Ran 等人，2019；Andor 等人，2019；Cobbe 等人；Chen 等，2021；，2022）。最近的方法主要是通过上下文学习（In-Context Learning）来Lester 提高语言模型的推理能力（Brown 等人，2020；等人等人，20212022；WeiWang 等人，2021，；，2022）。其中最具代表性的是思维链（Chain-of-Thought，CoT）（Wei 等人，2022 年），它将解决样本问题的中间推理过程呈现给语言模型，大大增强了其推理能力。

使用数据库的 LLM。LLM 在生成代码（包括 Python 代码、Excel 的执行命令和数据库的结构化查询语言（SQL））方面的能力令人印象深刻（OpenAI，2023 年）。ChatExcel⁴使用 LLM 生成 Excel 执行命令，简化了用户交互过程。BINDER（Cheng 等人，2022 年）提出了一个框架，将任务输入映射为编程语言（如 Python 代码）中的可执行程序，并与 API 绑定，以调用 LLMs 执行各种功能。SQL-PALM（Sun 等人，2023 年）提出了一种基于 LLM 的文本到 SQL 模型，使用基于执行的自洽提示方法，其性能大大优于以前的文本-2-SQL 方法。虽然以前的工作在一定程度上涉及数据库，但我们提出的 ChatDB 系统与这些方法有很大不同。具体来说，ChatDB 将数据库视为 LLM 的外部符号记忆模块，然后利用数据库读写基本数据信息，通过记忆增强推理过程，从而获得更准确的推理结果。

使用工具的 LLM。从工具使用的角度来看，ChatDB 也可以被看作是一种利用数据库作为工具的 LLM（Schick 等人，2023 年；Shen 等人，2023 年；Suris 等人，2023 年；Paranjape 等人，2023 年）。Toolformer（Schick 等人，2023 年）通过一系列演示，指示语言模型可以调用一些 API，利用外部工具解决当前问题。另一个代表作是 Auto-GPT⁵，它能让语言模型利用搜索引擎完成一系列令人印象深刻的任务。ChatDB 将数据库作为外部工具，其优势在于可以让语言模型保持更准确的记录并使用历史数据，从而解决更复杂的问题，尤其是那些需要准确历史数据进行推理的问题。

3 聊天数据库

在本节中，我们首先简要介绍任务的定义和设置。然后，我们将描述我们提出的 ChatDB 的整体框架。最后，我们将深入探讨作为 ChatDB 主要组成部分的内存链方法的细节。

3.1 任务定义

如果用户输入的是自然语言和数据库中现有表的详细信息（如果没有现有表，则不需要），那么我们的目标就是操作符号存储器，与外部数据库，以满足用户的要求。例如，如果用户（如商店经理）的命令是记录、修改、查询和删除特定数据，则相应的 SQL 操作应分别是插入、更新、选择和删除相应表中的相关数据。这些操作通常涉及数据库中的多个表。

3.2 框架概述

ChatDB 框架由三个主要阶段组成：输入处理、内存链和响应汇总，如图 2 所示。算法 1 详细说明了 ChatDB 响应用户输入的整个算法过程。

³ <https://github.com/Significant-Gravitas/Auto-GPT> ⁴ <https://chatexcel.com/>

⁵ <https://github.com/Significant-Gravitas/Auto-GPT>

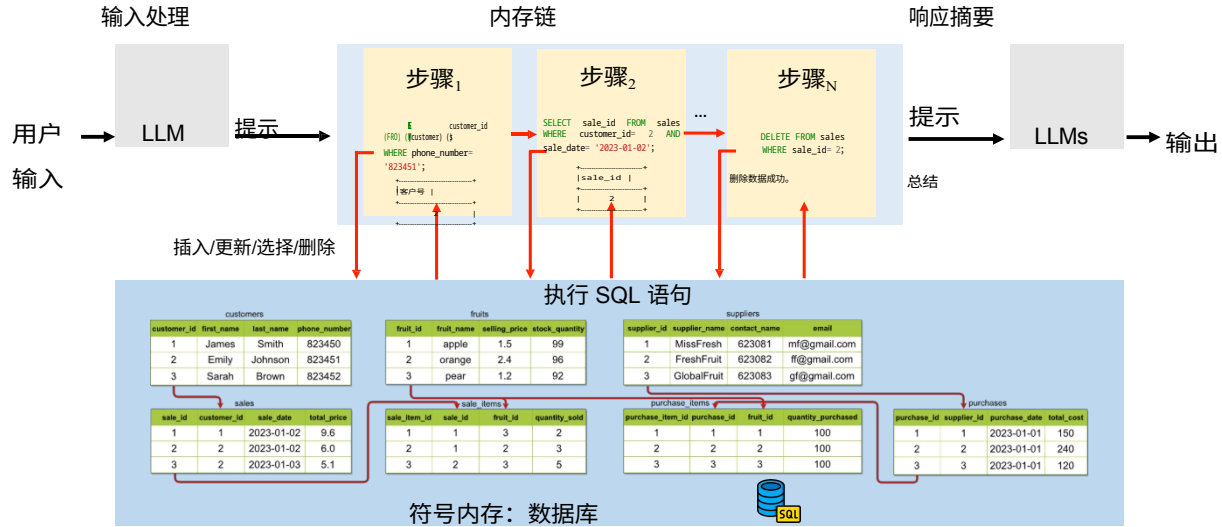


图 2: ChatDB 框架。红色箭头线 代表内存链的流程，表示多个内存操作之间的联系。数据库表之间的红色箭头线代表主键和外键之间的引用关系，即从主键到外键。为简洁起见，只显示了每个表的前四列。本例展示了电话号码为 823451 的客户退回 2023-01-02 购买的商的过程。

输入处理。如果回应用户输入需要使用内存，ChatDB 会生成一系列中间步骤，利用 LLM 来操作符号内存。否则，我们会直接使用 LLM 生成回复。

内存链。ChatDB 会执行一系列中间内存操作步骤来与符号内存交互。ChatDB 会根据之前生成的一系列 SQL 语句（包括插入、更新、选择、删除等操作）依次操作符号内存。外部数据库执行相应的 SQL 语句，更新数据库并返回结果。值得注意的是，在执行该操作前，ChatDB 会根据之前 SQL 语句的结果来决定是否更新内存操作步骤。ChatDB 会按照相同的步骤执行下一步，直到内存上的所有操作都完成为止。

响应总结。ChatDB 会根据一系列内存链步骤的结果，向用户总结最终的响应。

3.3 记忆链

思维链（Wei 等人，2022 年）强调将复杂的推理分解为一系列中间步骤。记忆链（CoM）通过提供一种符号记忆机制来支持与这些中间步骤相关的存储，可被视为一种增强思维链的方法。

内存链的目的是增强 LLM 在处理符号内存时的推理能力和稳健性。这种方法将用户输入转换为一系列中间内存操作，使 LLM 能够更准确、更有效地以符号方式操作内存。对于涉及与历史数据进行复杂而精确交互的实际应用（如管理环境中的记录保存和数据分析）来说，操作符号内存的能力尤为重要。

为了提高我们方法的性能和稳定性，我们采用了上下文学习（Brown 等人，2020 年），提供了若干记忆链步骤序列的提示范例，以及思维链提示。稳健而准确的记忆链过程使 LLM 能够更好地对符号记忆进行推理，并处理更复杂的场景。

内存链有两方面的优势。首先，与符号内存相比，它能使 LLM 更准确地执行复杂的数据库操作，增强其多跳推理能力。其次，通过将复杂操作分解为一系列中间内存操作，内存链方法增强了 LLM 处理复杂多表交互的能力。这种方法能让 LLM 更好地处理边缘情况和意外情况，因此在实际应用中是一种很有前途的方法。

算法 1 ChatDB 的算法**输入：**用户输入、数据库**输出：**回复

```

1: 如果需要操作内存来回复用户输入，那么
2:   memOps= LLMgetSteps(userInput)
3: 否则
4:   reply= LLM(userInput)
5:   return reply
6: end if

7: sqlResults= []
8: newMemOps= []
9: for each memOp in memOps do
10:   if need update memOp based on sqlResults then
11:     newMemOp= LLMupdateOperation(memOp, sqlResults)
12:   否则
13:     newMemOp= memOp
14:   end if
15:   sqlResult= executeOperation(newMemOp, dataBase)
16:   sqlResults.append(sqlResult)
17:   newMemOps.append(newMemOp)
18: 结束 for

LLM(summary)(userInput, newMemOps, sqlResults)
reply

```

▷开始处理输入

▷ 使用 LLM 生成中间步骤

▷直接使用 LLM 生成回复

▷内存链开始

▷在数据库上执行操作

▷ 开始 回复 摘要 19: reply=

▷ 总结 最终 回复 20: **return**

3.4 与以前的内存扩充 LLM 比较

表 1：与基于提示的内存和基于矩阵的内存的比较。

类型	模型	内存格式	支持的操作	内存存储	内存执行	可解释性	状态跟踪
符号 据库	聊天数	符号存储（如数据 库）	插入、删除、 更新、选择	结构式	符号	高	是
基于提示	自动 GPT	内容及其 向量嵌入	插入、选择	半结构化	非符号	正常	无
阵	RMT	内存矩阵(标记/)	读、写	半结构	非符号	低	有

在本小节中，我们将全面比较 ChatDB 和最近使用内存模块增强基于 Transformer 的语言模型的方法。以往工作中提出的语言模型记忆模块可大致分为两类。第一类内存存储上下文，并使用检索模型从过去的交互中找到与当前对话最相关的内容，然后将其作为语言模型的提示（Khattab 等人，2022 年）。我们将这种类型的记忆称为*基于提示的记忆*。第二种方法利用额外的记忆标记或记忆矩阵作为记忆（Bulatov 等人，2022 年），我们称之为*基于矩阵的记忆*。我们基于以下几个方面将 ChatDB 与这些方法进行比较：

1. 存储器格式。这一方面与内存的存储格式有关。ChatDB 使用数据库作为内存。*基于提示的记忆*（Park 等人，2023 年）存储相关的交互内容和/或其相应的向量嵌入。*基于矩阵的记忆*则采用了额外的可训练记忆标记（Bulatov 等人，2022，2023）或可训练记忆矩阵（Graves 等人，2014）。

2. 支持的操作。这方面指的是操作存储器所支持的操作。ChatDB 支持插入、删除、更新和选择数据库内存中的数据等操作。*基于提示的内存*主要支持插入和选择操作，但缺乏对更新和删除的完整支持。*基于矩阵的内存*支持读取（选择）和写入（插入、更新、删除）操作。不过，神经网络执行的具体操作并不明确。

3. 内存存储。这方面指的是数据存储在内存中的格式，特别是是否结构化。ChatDB 使用数据库以结构化格式存储内存，而基于提示的内存

和*基于矩阵的记忆*被视为半结构化的。原因在于，向量嵌入和记忆矩阵有特定的维度和大小，但每个维度并不具有具体明确的含义。

4. 内存执行。这方面的重点是内存操作的执行方式，特别是它们是否具有符号性。ChatDB 使用 SQL 对数据库内存执行操作，而 SQL 是一种符号语言，因此 ChatDB 本身就是符号语言。*基于提示的内存*使用向量嵌入根据相似度量进行选择，并使用语言编码器获取向量嵌入进行插入。这两者都被视为非符号执行。在*基于矩阵的增强型 LLM* 内存中，内存操作完全由神经网络控制，因此也属于非符号执行。

5. 可解释性。这方面指的是内存的可解释程度。在 ChatDB 中，内存是以结构化的明确格式存储的，其操作是符号化的，因此可解释性很高。在*基于提示的内存*中，由于在解释向量嵌入方面存在固有的挑战，可解释性通常是有限的。对于*基于矩阵的记忆*方法，由于记忆完全由神经网络隐式控制，因此可解释性较低。

6. 状态跟踪。这方面指的是内存是否能有效跟踪 LLM 的当前状态。就 ChatDB 而言，它的内存能够准确跟踪 LLM 的当前状态。以水果店实验为例，在处理完每条记录后，ChatDB 的数据库内存都会更新，以反映水果店的最新状态。这展示了 ChatDB 的内存是如何有效跟踪其当前状态的。由于采用了符号内存执行方式，ChatDB 的内存可以轻松回滚到任何需要的时间戳，从而提供了更大的灵活性和可控性。在*基于矩阵的内存*方法中，内存是由模型本身不断更新和改变的，因此它能跟踪 LLM 的当前状态。然而，*基于提示的记忆*方法只是存储历史背景，只知道过去发生了什么，而不清楚当前的状态。

通过对这些方面的研究，我们发现了 ChatDB 与现有方法相比的独特之处和能力。ChatDB 的优越性凸显了使用符号记忆增强 LLM 的优势。

4 评估

在本节中，我们将通过实验来评估使用数据库作为符号存储器来增强 LLM 的效果。实验结果表明，ChatDB 明显优于基线模型 ChatGPT，突出了符号存储器集成的优势。

4.1 实验设置

如前所述，使用数据库作为符号存储器尤其适用于需要精确记录和处理历史信息的场景，例如各种数据管理场景。为了适应 ChatDB 的使用情况，并与其他模型进行定量比较，我们构建了一个模拟水果店管理的合成数据集。

此外，为了评估模型的性能，我们还收集了一组 50 个问题，并标注了标准答案。这些问题难度不一，有需要多跳推理的难题，也有只需要从历史数据中检索信息的简单问题。其中有 15 道简单题和 35 道难题。每个问题都由模型独立回答。

4.1.1 模型配置

聊天数据库。ChatDB 使用的 LLM 是 ChatGPT (GPT-3.5 Turbo)，超参数 *温度* 设置为 0。我们使用 MySQL 数据库作为外部符号存储器。

基线。我们使用 ChatGPT (GPT-3.5 Turbo) 作为基线模型，最大标记长度为 4096。与 ChatDB 类似，我们将 *温度* 设置为 0。

4.1.2 数据集

我们合成了一个水果店管理记录数据集，称为“水果店数据集”。该数据集模拟了水果店的四种常见操作：采购、销售、价格变更和退货。我们确保所有历史记录都是有效的，不会出现负库存等问题。我们生成了 70 条按时间顺序排列的记录，总计约 3.3k 个代币，在 ChatGPT 的最大代币长度限制（4096 个代币）之内。

(a) 购买商品：商店从供应商处购买商品。

记录：我们于 2023-01-01 日从 "ABC" 公司 (abc_sup@example.com, 10080) 新进了一批水果。采购数量包括 24 公斤樱桃，单价为 0.8。我们打算将樱桃的单价定为 1.3 美元。

ChatDB:

开始与外部数据库（符号存储器）交互。步骤 1：如果不存在，则插入供应商 "ABC"

```
INSERT INTO suppliers (supplier_name, contact_number, email)
VALUES ('abc', '10080', 'abc_sup@example.com')
WHERE NOT EXISTS (SELECT 1 FROM suppliers WHERE supplier_name = 'abc');
```

步骤 2：如果不存在，则插入水果（将售价设为 NULL，库存量设为 0）

```
INSERT INTO fruits (fruit_name, selling_price, stock_quantity,
  fruit_type, shelf_life)
VALUES ('cherry', NULL, 0, NULL, NULL)
WHERE NOT EXISTS (SELECT 1 FROM fruits WHERE fruit_name = 'cherry');
```

步骤 3：插入采购信息

```
INSERT INTO purchases (supplier_id, purchase_date,
  total_cost)
VALUES ((SELECT supplier_id FROM suppliers WHERE
  supplier_name = 'ABC'), '2023-01-01', 24 * 0.8);
```

步骤 4：插入采购项目

```
INSERT INTO purchase_items (purchase_id, fruit_id,
  quantity_purchased, cost_per_item, item_total_cost)
VALUES ((SELECT MAX(purchase_id) FROM purchases), (SELECT fruit_id
  FROM fruits WHERE fruit_name = 'apple'), 9, (SELECT
  selling_price FROM fruits WHERE fruit_name = 'apple'),
  (SELECT selling_price FROM fruits WHERE fruit_name = 'apple') * 9);
```

步骤 5：更新樱桃的库存数量

```
UPDATE fruits
SET stock_quantity = stock_quantity + 24 WHERE
fruit_name = 'cherry';
```

步骤 6：如果给出了新售价，更新樱桃的售价

```
UPDATE fruits
SET selling_price = 1.3 WHERE
fruit_name = 'cherry';
```

(c) 退货：顾客将购买的商品带回商店。

记录：由于顾客退回了购买的商品，我们需要撤销顾客 "John Doe"（电话：123-456-7890，电子邮件：john.doe@example.com）在 2023-01-05 日进行的销售交易。

聊天数据库:

开始与外部数据库（符号存储器）交互。步骤 1：查找该客户在该日期的 sale_id

```
SELECT sale_id FROM sales
WHERE customer_id = (SELECT customer_id FROM customers WHERE
  phone_number = '123-456-7890' AND email = 'john.doe@example.com')
AND sale_date = '2023-01-05';
```

数据库响应:

sale_id
9

步骤 2：获取这次销售的所有 fruit_id 和 quantity_sold，将 <sale_id> 替换为前面查询的结果

```
SELECT fruit_id, quantity_sold FROM sale_items
WHERE sale_id = 9 [<sale_id>];
```

数据库响应:

fruit_id	quantity_sold
7	10
6	9

(b) 销售商品：顾客从商店购买商品

记录：2023-01-02 向 "Bob Smith"（联系方式：123-456-7893，bob.smith@example.com）。出售了商品购买的商品是 9 千克苹果、4 千克樱桃。

ChatDB:

开始与外部数据库（符号存储器）交互。步骤 1：插入客户 "Bob Smith"（如果不存在）

```
INSERT INTO customers (first_name, last_name, phone_number,
  email)
VALUES ('Bob', 'Smith', '123-456-7893', 'bob.smith@example.com')
WHERE NOT EXISTS (SELECT 1 FROM customers WHERE phone_number = '123-456-7893');
```

步骤 2：插入销售额

```
INSERT INTO sales (customer_id, sale_date, total_price)
VALUES ((SELECT customer_id FROM customers WHERE phone_number = '123-456-7893'), '2023-01-02', (SELECT selling_price
  FROM fruits WHERE fruit_name = 'apple') * 9 + (SELECT
  selling_price FROM fruits WHERE fruit_name = 'cherry') * 4);
```

步骤 3：插入销售项目

```
INSERT INTO sale_items (sale_id, fruit_id, quantity_sold,
  price_per_item, item_total_price)
VALUES ((SELECT MAX(sale_id) FROM sales), (SELECT fruit_id
  FROM fruits WHERE fruit_name = 'apple'), 9, (SELECT
  selling_price FROM fruits WHERE fruit_name = 'apple'),
  (SELECT selling_price FROM fruits WHERE fruit_name = 'apple') * 9),
  ((SELECT MAX(sale_id) FROM sales), (SELECT fruit_id
  FROM fruits WHERE fruit_name = 'cherry'), 4, (SELECT
  selling_price FROM fruits WHERE fruit_name = 'cherry'),
  (SELECT selling_price FROM fruits WHERE fruit_name = 'cherry') * 4);
```

步骤 4：更新苹果和樱桃的库存量

```
UPDATE fruits
SET stock_quantity = CASE
  WHEN fruit_name = 'apple' THEN stock_quantity - 9
  WHEN fruit_name = 'cherry' THEN stock_quantity - 4
  ELSE stock_quantity
WHERE fruit_name IN ('apple', 'cherry');
```

结束

步骤 3：增加本次销售中售出的每种水果的库存数量，将 <quantity_sold> 替换为之前查询的结果

```
UPDATE fruits
SET stock_quantity = stock_quantity + 10 [<quantity_sold>]
WHERE fruit_id = 7 [<fruit_id>];
```

更新水果

```
SET stock_quantity = stock_quantity + 9 [<quantity_sold>]; UPDATE fruits
WHERE fruit_id = 6 [<fruit_id>];
```

步骤 4：删除本次销售的销售商品，将 <sale_id> 替换为之前查询的结果

```
DELETE FROM sale_items WHERE sale_id = 9 [<sale_id>];
```

步骤 5：删除销售记录，用前面的查询结果替换 <sale_id>

```
DELETE FROM sales WHERE sale_id = 9 [<sale_id>];
```

(d) 更改价格：修改商品价格。

记录：2023-01-05 日，商店中梨的销售价格改为 1.6 美元/个。

ChatDB:

开始与外部数据库（符号存储器）交互。步骤 1：更新梨的销售价格

更新水果

```
SET selling_price = 1.6 WHERE
fruit_name = 'pear';
```

图 3：示例展示了 ChatDB 对水果商店数据集中记录的四种常见操作的响应。"9 [<sale_id>]" 表示在更新的 SQL 语句中

<sale_id> 被替换为 9。由于页面大小限制，省略了一些不必要的细节和 ChatDB 的摘要。

假设你是一家水果店的经理，擅长分析历史记录。

该水果店于 2023 年 1 月 1 日新开业。给定 2023 年 1 月水果店的历史记录，其中包括顾客姓名、交易日期、水果价格、购买数量和是否退货，你需要回答一些问题。

默认情况下，在进行计算时应排除已退货的销售交易。

以下是水果店的历史记录，这些记录根据发生时间按时间顺序排列，周围有三个回车键：

```
...
{记录}
...
```

根据历史记录，回答有关水果店的问题：
问题

图 4：提示 ChatGPT 回答 水果店数据集中的问题。实际使用时，占位符 "记录" 和 "问题" 将被具体细节取代。

为什么要限制数据集的标记长度？ 如果数据集的标记长度超过了 ChatGPT 的最大标记长度，就需要进行记忆。然而，基于向量嵌入的主流记忆检索方法容易出错。这不可避免地会导致 ChatGPT 性能下降，这是不可取的。因此，我们特意将数据集的标记长度设计在 ChatGPT 的最大标记长度范围内，以避免使用内存，最大限度地提高模型的性能。请注意，ChatDB 的性能通常不受数据集标记长度的影响。因此，如果当数据集较小时，ChatDB 的性能优于 ChatGPT，那么当数据集较大时，ChatDB 的性能也会优于增加内存的 ChatGPT。

4.1.3 处理记录

对于 ChatDB 来说，第一步是初始化数据库。我们需要为特定任务场景生成合理的数据库模式，并在数据库中创建表格。数据库模式的生成可以手动完成，也可以使用 LLM。接下来，对于数据集中的每条记录，ChatDB 都会逐一进行处理。ChatDB 使用 LLM 控制器，按照算法 1 操作外部数据库（*即*符号内存）。如图 3 所示，我们举例说明了 ChatDB 对水果店数据集中四种常见操作的响应，即购买、销售、更改价格和退货。值得强调的是，ChatDB 是逐条处理记录的，因此对记录总数并不敏感。此外，ChatDB 中数据库操作的每一步都是符号化的，不会出错。因此，从理论上讲，ChatDB 可以在不影响性能的情况下处理无限多的历史记录。不过，对于 ChatGPT 或现有的内存增强型 LLM 来说，过长的历史记录会大大降低性能。在本实验中，与 ChatGPT 基线一样，由于记录并不长，我们只是将其视为提示的一部分。

4.1.4 回答问题

在回答问题时，ChatDB 不再需要记录作为提示的一部分。在处理完记录后，信息会存储在符号内存中。按照算法 1，ChatDB 利用 SQL 语句来执行一系列数据库查询（包括计算），从而回答问题。另一方面，ChatGPT 将记录作为提示的一部分，并直接提出问题。提示模板如图 4 所示。

4.2 结果

实验结果如表 2 所示，结果清楚地表明 ChatDB 的准确率明显高于 ChatGPT。虽然 ChatGPT 能够回答简单的问题，但在处理需要多跳推理和精确计算的难题时，它就显得力不从心了。因此，ChatGPT 对这些难题的准确率很低。相比之下，ChatDB 的准确率明显较高，突出了利用数据库作为符号存储器的优势。这种方法不仅能防止错误积累，还能增强 LLM 的多跳推理和精确计算能力。

我们在图 5 中展示了两个模型回答问题的几个例子，以进行比较。在所有这些例子中，ChatDB 都能正确回答问题，而 ChatGPT 则失败了。如图 5 (a) 所示，ChatGPT 在计算每笔销售交易的总价时经常出现错误。有时，公式是正确的，但计算是错误的，而有时，甚至连公式都是错误的。此外，ChatGPT 很难找到所有有效的销售交易。

(a) 问题2023 年 1 月的总收入是多少？

ChatGPT:

要计算 2023 年 1 月的总收入，我们需要汇总每笔销售交易产生的收入。我们可以使用以下公式计算每笔交易的收入：收入 = 数量 * 售价 我们还需要排除已退回的销售交易。根据历史记录，以下销售交易已被退回：

- Alice Smith, 2023-01-17 - Bob Smith, 2023-01-05 [...]

因此，在计算总收入时，我们需要剔除这些交易。下面是每笔销售交易的计算结果：

- 鲍勃-史密斯, 2023-01-02: (9 公斤 * 3.8 美元/公斤)+(4 公斤 * 1.3 美元/公斤)= 39.1 美元 [错误]: 计算错误, 应为 39.4 美元

- Sue Davis 在 2023-01-02: 1 公斤 * 3.8 美元/公斤 = 3.8 美元

- Bob Smith 于 2023-01-05: (5 公斤 * 1.8 美元/公斤)+(2 公斤 * 3.0 美元/公斤)+(6 公斤 * 2.8 美元/公斤)

* 2.8 USD/kg)= 47.6 USD [错误]: 计算错误, 此交易已被退回。

[...][错误]: 大部分销售交易计算有误。[STOP]: 因为超过了最大令牌长度。

。

ChatDB:

开始与外部数据库（符号内存）交互。步骤 1: 计算 2023 年 1 月的总收入

```
SELECT SUM(total_price) AS total_revenue
FROM sales
WHERE sale_date >= '2023-01-01' AND sale_date < '2023-02-01';
```

数据库响应:

总收入
707.0

(b) 问题2023 年 1 月的哪一天收入最高？ ChatGPT:

要计算每天的收入，我们需要将当天的销售总额（不包括退货商品）相加，然后乘以售价。然后，我们可以比较每天的收入，以确定哪一天的收入最高。

以下是每天的销售明细：

- 1 月 2 日: 9 千克苹果售价 3.8= 34.2, 4 千克樱桃售价 1.3= 5.2, 3 千克樱桃售价 1.3= 3.9, 3 千克橙子 2.1= 6.3, 总销售额= 49.6 [错误]: 应为 43.2

- 1 月 4 日: 2 千克苹果售价 1.5= 3, 4 千克梨售价 1.6= 6.4, 7 千克樱桃售价 1.3= 9.1

, 总销售额 = 18.5 [错误]: 应为 27.9

- 1 月 5 日: 5 公斤樱桃, 售价 1.9= 9.5, 10 公斤橙子, 售价 1.8= 18, 1 公斤樱桃, 售价 1.9

=1.9, 1 公斤葡萄, 售价 3.2= 3.2, 总销售额= 32.6 [错误]: 应为 83

- 1 月 6 日: 3 千克苹果 1.5= 4.5, 10 千克葡萄 3.2= 32, 总销售额 = 36.5 [错误]: 应为 29.4

- 1 月 7 日: 4 公斤梨, 售价 1.6= 6.4, 总销售额= 6.4 [错误]: 应为 37.2

- 1 月 8 日: 6 公斤桃, 价格 3= 18, 总销售额= 18 [错误]: 应为 21.6

[...][错误]: 大部分每日收入计算错误。[STOP]: 因为超过了最大令牌长度。

ChatDB:

开始与外部数据库（符号内存）交互。步骤 1: 计算 2023 年 1 月每天的总收入

```
SELECT sale_date, SUM(total_price) AS revenue
FROM sales
WHERE sale_date >= '2023-01-01' AND sale_date < '2023-01-31'
GROUP BY sale_date;
```

数据库响应:

sale_date	revenue
2023-01-02	43.2
2023-01-04	27.9
...	...

第二步: 找出收入最高的一天

```
SELECT sale_date, SUM(total_price) AS revenue
FROM sales
WHERE sale_date >= '2023-01-01' AND sale_date < '2023-01-31'
GROUP BY sale_date ORDER BY revenue DESC LIMIT 1;
```

数据库响应:

sale_date	revenue
2023-01-30	119.7

(c) 问题2023 年 1 月每笔销售交易的平均水果重量是多少？

ChatGPT: 要计算每笔销售交易的水果平均重量，我们首先需要确定 2023 年 1 月的所有销售交易，并计算出这些交易中售出的水果总重量。然后，我们可以用总重量除以

的数量，从而得到每笔交易的平均重量。以下是 2023 年 1 月的销售交易：

- 鲍勃-史密斯 2023-01-02: 9 千克苹果, 4 千克樱桃

- Sue Davis (2023-01-02): 1 千克苹果

- 鲍勃-史密斯 (2023-01-05): 5 千克橘子、2 千克桃子、6 千克梨 [错误]: 这个

事务已返回。

- Sue Davis 于 2023-01-05: 5 千克梨、10 千克橙子、1 千克樱桃

- 爱丽丝-史密斯在 2023-01-05: 5 千克樱桃

[...][错误]: 没有找出所有有效的销售交易。

要计算售出水水果的总重量，我们可以将上述交易中售出的每种水果的数量相加：

- 苹果 16 千克 - 樱桃 12 千克 - 橙 10 千克 - 梨 18 千克 - 猕猴桃: 12 千克

葡萄: 18 千克 - 香蕉: 27 千克 - 草莓 7 公斤 - 桃子桃子: 9 千克 - 李子: 8 千克

2023 年 1 月售出的水果总重量为 127 千克。[错误]: 应该是

278.即使根据上面的计算, 总重量也应该是 137 千克, 而不是 127 千克。

要计算每笔销售交易的平均重量，我们需要用总重量除以销售交易数量。2023 年 1 月有 18 笔销售交易，[错误]: 所以平均权重 [停止]: 因为超过了最大标记长度。

聊天数据库:

开始与外部数据库（符号内存）交互。步骤 1: 计算 2023 年 1 月售出水水果的总重量

```
SELECT SUM(si.quantity_sold) AS
  total_weight
FROM sale_items si
JOIN sales s ON si.sale_id = s.sale_id
WHERE s.sale_date BETWEEN '2023-01-01'
  AND '2023-01-31';
```

数据库响应:

总重量
278

步骤 2: 计算 2023 年 1 月的销售交易数量

```
SELECT COUNT(DISTINCT s.sale_id) AS
  num_sales
FROM sales s
WHERE s.sale_date BETWEEN '2023-01-01'
  AND '2023-01-31';
```

数据库响应:

num_sales
28

步骤 3: 计算 2023 年 1 月每笔销售交易的水果平均重量，将 <total_weight> 和 <num_sales> 替换为之前查询的结果

```
SELECT 278 [< total_weight >] / 28
  AS avg_weight_per_sale;
```

数据库响应:

每次销售的平均重量
9.93

图 5: ChatDB 和 ChatGPT 回答问题的示例。ChatGPT 未能正确回答任何问题，而 ChatDB 则成功回答了所有问题。ChatGPT 的

问题用红色标出。为简洁起见，省略了一些无关细节和 ChatDB 的总结。灰色省略号[.....]表示已对回答进行了删减。

表 2: 水果店数据集中问题解答的实验结果。共有 50 个问题, 其中 15 个为简单问题, 35 个为困难问题。

模型	简单	难	全部	精确度
聊天 GPT	10/15	1/35	11/50	22%
聊天数据库 (我们的)	13/15	28/35	41/50	82%

交易, 导致其回答过程出现错误。这个问题在所有这些示例中都很常见, 也很明显。此外, ChatGPT 往往会连续出错, 导致大量错误累积。

相比之下, ChatDB 在这些示例中表现相当出色。在最初处理记录时, 会应用符号操作 (即 SQL 操作) 来操作数据库 (即符号内存), 确保所有信息都以结构化的形式存储在数据库中。在回答问题时, ChatDB 会生成 SQL 语句来查询数据库。这三个例子分别展示了 ChatDB 在解决需要一个、两个和三个内存链步骤的问题时的有效性。我们可以观察到, ChatDB 准确地回答了问题, 而且内存链的执行逻辑非常清晰, 每一步都紧密相连并接近最终答案。从这些例子中, ChatDB 的优势体现在两个方面:

1. 通过内存链方法, 复杂问题被分解成多个内存操作步骤, 从而简化了问题的复杂性。每个步骤的结果都会作为中间结果被精确存储, 并在后续步骤中使用, 这大大有助于复杂推理。
2. 符号内存可实现精确操作和计算。ChatDB 通过执行 SQL 语句将许多计算任务委托给外部数据库, 从而确保了每一步的准确性, 并防止错误累积。

总之, 通过利用外部数据库作为符号内存, ChatDB 在本实验中的表现明显优于 ChatGPT。

5 结论

在本文中, 我们介绍了 ChatDB, 这是一个以数据库形式用符号记忆增强 LLM 的框架。我们展示了符号内存和内存链方法在增强复杂推理和防止错误积累方面的优势和能力。通过为中间结果提供精确的存储机制, 符号存储器可实现准确可靠的操作。此外, 符号语言 (如 SQL) 的使用允许对存储信息进行符号计算和操作。通过实验评估, 我们发现与 ChatGPT 相比, ChatDB 的性能有了显著提高。ChatDB 中符号存储器的集成大大提高了模型在管理设置中处理各种查询和推理任务的能力。这一改进凸显了在 LLM 中利用符号内存的好处和有效性。

参考文献

- Andor, D., He, L., Lee, K., and Pitler, E. (2019). 给伯特一个计算器: *ArXiv preprint arXiv:1909.00109*.
- Anil, R., Dai, A. M., Firat, O., Johnson, M., Lepikhin, D., Passos, A., Shakeri, S., Taropa, E., Bailey, P., Chen, Z., et al. (2023). *ArXiv preprint arXiv:2305.10403*.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). 语言模型是少量学习者. *神经信息处理系统进展*, 33:1877-1901.
- Bulatov, A., Kuratov, Y., and Burtsev, M. (2022). Recurrent memory transformer. *神经信息处理系统进展*, 35:11079-11091.
- Bulatov, A., Kuratov, Y., and Burtsev, M. S. (2023). *ArXiv preprint arXiv:2304.11062*.
- Chen, A., Phang, J., Parrish, A., Padmakumar, V., Zhao, C., Bowman, S. R., and Cho, K. (2023). *ArXiv preprint arXiv:2305.14279*.

- Chen, W., Ma, X., Wang, X., and Cohen, W. W. (2022).思维提示程序: *ArXiv preprint arXiv:2211.12588*.
- Cheng, Z., Xie, T., Shi, P., Li, C., Nadkarni, R., Hu, Y., Xiong, C., Radev, D., Ostendorf, M., Zettlemoyer, L., et al. (2022).*ArXiv preprint arXiv:2210.02875*.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. (2021).*ArXiv preprint arXiv:2110.14168*.
- Du, Z., Qian, Y., Liu, X., Ding, M., Qiu, J., Yang, Z., and Tang, J. (2022).Glm: 自回归空白填充通用语言模型预训练。第60届计算语言学协会年会论文集》(第1卷: 长篇论文), 第320-335页。
- Graves, A., Wayne, G., and Danihelka, I. (2014).*ArXiv preprint arXiv:1410.5401*.
- Guu, K., Lee, K., Tung, Z., Pasupat, P., and Chang, M. (2020).检索增强语言模型预训练。在机器学习国际会议, 第3929-3938页。PMLR.
- Hao, S., Gu, Y., Ma, H., Hong, J. J., Wang, Z., Wang, D. Z., and Hu, Z. (2023).*arXiv preprint arXiv:2305.14992*.
- Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., and Grave, E.(2022).使用检索增强语言模型的少量学习。 *arXiv preprint arXiv:2208.03299*.Johnson, D. D. (2017).学习图形状态转换。在 学习表征国际会议上。
- Khattab, O., Santhanam, K., Li, X. L., Hall, D., Liang, P., Potts, C., and Zaharia, M. (2022).演示-搜索-预测: *ArXiv preprint arXiv:2212.14024*.
- Lester, B., Al-Rfou, R., and Constant, N. (2021). *ArXiv preprint arXiv:2104.08691*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. (2020).知识密集型 NLP 任务的检索增强生成。 *神经信息处理系统进展*, 33:9459-9474。
- OpenAI (2023).Gpt-4 technical report.
- Paranjape, B., Lundberg, S., Singh, S., Hajishirzi, H., Zettlemoyer, L., and Ribeiro, M. T. (2023).艺术: *ArXiv preprint arXiv:2303.09014*.
- Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023).Generative agents: *arXiv preprint arXiv:2304.03442*.
- Pie, kos, P., Michalewski, H., and Malinowski, M. (2021). *ArXiv preprint arXiv:2106.03921*.
- Ran, Q., Lin, Y., Li, P., Zhou, J., and Liu, Z. (2019).Numnet: *ArXiv preprint arXiv:1910.06701*.
- Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023).Toolformer: *ArXiv preprint arXiv:2302.04761*.
- Schuermans, D. (2023).内存增强的大型语言模型在计算上是通用的。 *arXiv preprint arXiv:2301.04589*.
- Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. (2023).Hugginggpt: *ArXiv preprint arXiv:2303.17580*.
- Sun, R., Arik, S. O., Nakhost, H., Dai, H., Sinha, R., Yin, P., and Pfister, T. (2023).Sql-palm: 改进的文本到 sql 大语言模型适应。
- Surís, D., Menon, S., and Vondrick, C. (2023).Vipergpt: *arXiv preprint arXiv:2303.08128*.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). *ArXiv preprint arXiv:2302.13971*.

Wang, W., Chen, Z., Chen, X., Wu, J., Zhu, X., Zeng, G., Luo, P., Lu, T., Zhou, J., Qiao, Y., et al. (2023). Visionllm: 大型语言模型也是以视觉为中心任务的开放式解码器。 *arXiv 预印本 arXiv:2305.11175*.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., and Zhou, D. (2022). 自洽性提高了语言模型的思维链推理能力。

Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. (2021). *ArXiv preprint arXiv:2109.01652*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., and Zhou, D. (2022). 大型语言模型中的思维链提示推理。 *arXiv preprint arXiv:2201.11903*.

Wu, Y., Rabe, M. N., Hutchins, D., and Szegedy, C. (2022a). 记忆变换器。 *arXiv preprint arXiv:2203.08913*.

Wu, Y., Zhao, Y., Hu, B., Minervini, P., Stenetorp, P., and Riedel, S. (2022b). 用于知识密集型 NLP 任务的高效记忆增强变换器。 *arXiv 预印本 arXiv:2210.16773*.

Zeng, A., Liu, X., Du, Z., Wang, Z., Lai, H., Ding, M., Yang, Z., Xu, Y., Zheng, W., Xia, X., et al. (2022). Glm-130b: An open bilingual pre-trained model. *ArXiv preprint arXiv:2210.02414*.

Zhong, Z., Lei, T., and Chen, D. (2022). 用记忆增强训练语言模型。 *arXiv preprint arXiv:2205.12674*.